JAVA EXCEPTION HANDLING PROGRAMS REPORT

Clearer UML-style diagrams | Black text and lines only

Includes the nine submitted programs with aim, requirements, algorithm, UML diagram, source code, execution output, and result.

Contents

1. ArrayIndexExample
2. CustomExceptionExample
3. FileNotFoundExample
4. IOExceptionExample
5. NullPointerExample
6. NumberFormatExample
7. StringIndexExample
8. ClassCastExample
9. IllegalArgumentExample

Program 1: ArrayIndexExample

1. Aim

To demonstrate accessing array elements using valid index positions.

2. Requirements

Java Development Kit (JDK) and a text editor or IDE.

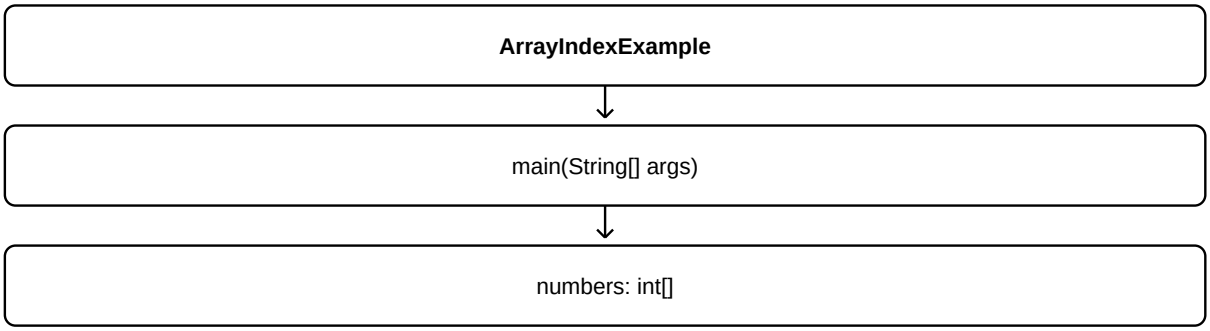
3. Algorithm

1. Create an integer array containing five values.

2. Print the first, third, and last elements using valid indexes.

4. UML Diagram (simplified)

Read from top to bottom. Boxes show the main class, method, action, or exception-handling step.



5. Source Code

```
public class ArrayIndexExample {
    public static void main(String[] args) {

        int[] numbers = {10, 20, 30, 40, 50};

        System.out.println("First Element: " + numbers[0]);
        System.out.println("Third Element: " + numbers[2]);
        System.out.println("Last Element: " + numbers[4]);
    }
}
```

6. Execution Output

```
First Element: 10
Third Element: 30
Last Element: 50
```

7. Result

The program displays array elements using valid indexes. It does not trigger or handle ArrayIndexOutOfBoundsException.

Program 2: CustomExceptionExample

1. Aim

To demonstrate creating and handling a user-defined checked exception.

2. Requirements

Java Development Kit (JDK) and a text editor or IDE.

3. Algorithm

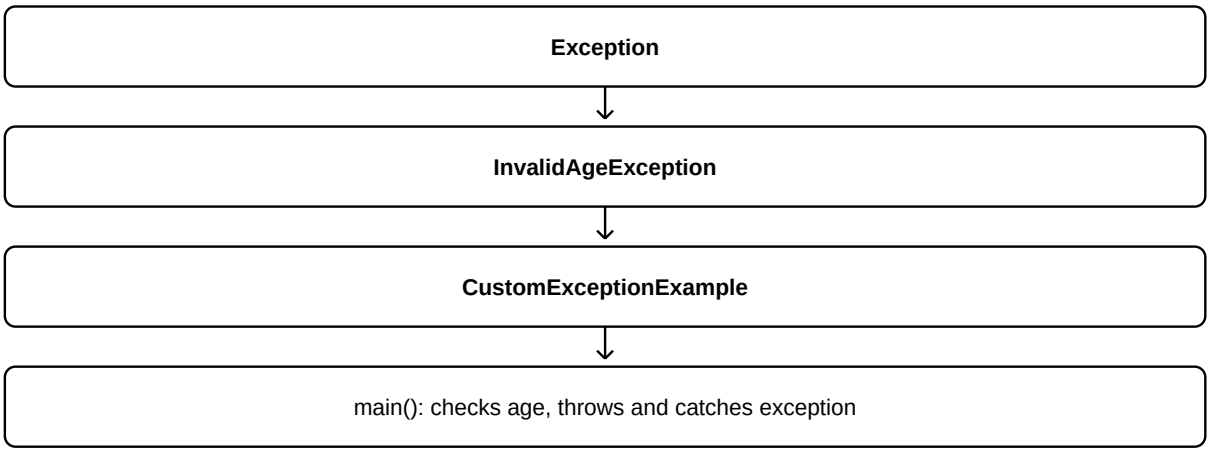
1. Define InvalidAgeException as a subclass of Exception.

2. If age is below 18, throw InvalidAgeException.

3. Catch the exception and print its message.

4. UML Diagram (simplified)

Read from top to bottom. Boxes show the main class, method, action, or exception-handling step.



5. Source Code

```
class InvalidAgeException extends Exception {

    InvalidAgeException(String message) {
        super(message);
    }
}

public class CustomExceptionExample {

    public static void main(String[] args) {

        int age = 16;

        try {
            if (age < 18) {
                throw new InvalidAgeException("Age must be 18 or above");
            }

            System.out.println("Eligible to vote");
        }
        catch (InvalidAgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

6. Execution Output

```
Exception: Age must be 18 or above
```

7. Result

InvalidAgeException is thrown when the age is below 18 and handled by the catch block.

Program 3: FileNotFoundExample

1. Aim

To demonstrate handling FileNotFoundException when opening a file.

2. Requirements

JDK, text editor or IDE, and student.txt for the successful path.

3. Algorithm

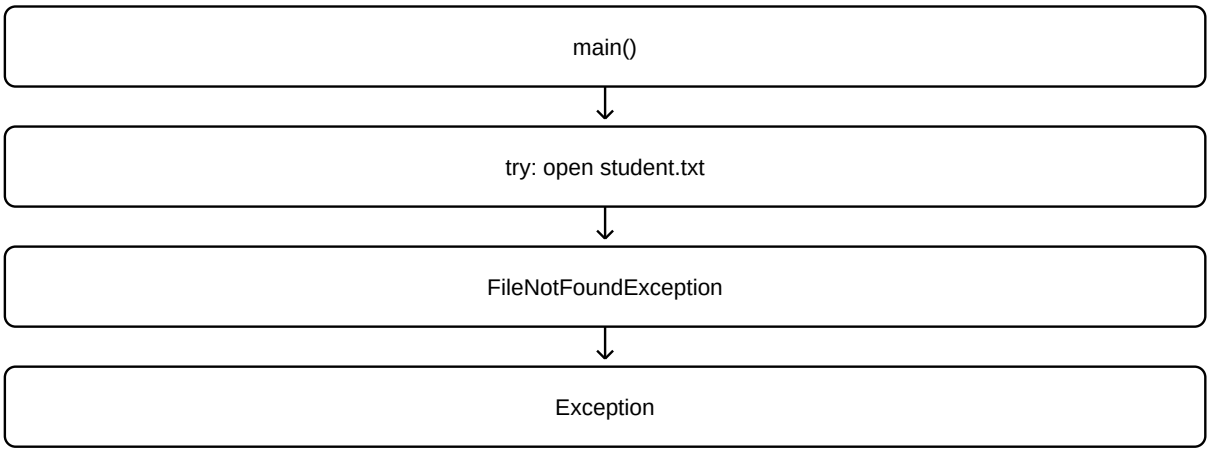
1. Try to open student.txt using FileReader.

2. If opening succeeds, print a message and close the file.

3. Catch FileNotFoundException if the file is missing; catch other exceptions separately.

4. UML Diagram (simplified)

Read from top to bottom. Boxes show the main class, method, action, or exception-handling step.



5. Source Code

```
import java.io.FileReader;
import java.io.FileNotFoundException;

public class FileNotFoundExample {
    public static void main(String[] args) {

        try {
            FileReader file = new FileReader("student.txt");

            System.out.println("File opened successfully");
            file.close();
        }
        catch (FileNotFoundException e) {
            System.out.println("File not found");
        }
        catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

6. Execution Output

```
File not found
(when student.txt does not exist in the working directory)
```

7. Result

The program handles a missing file using FileNotFoundException.

Program 4: IOExceptionExample

1. Aim

To demonstrate handling input/output errors while reading a text file.

2. Requirements

JDK, text editor or IDE, and data.txt for normal reading.

3. Algorithm

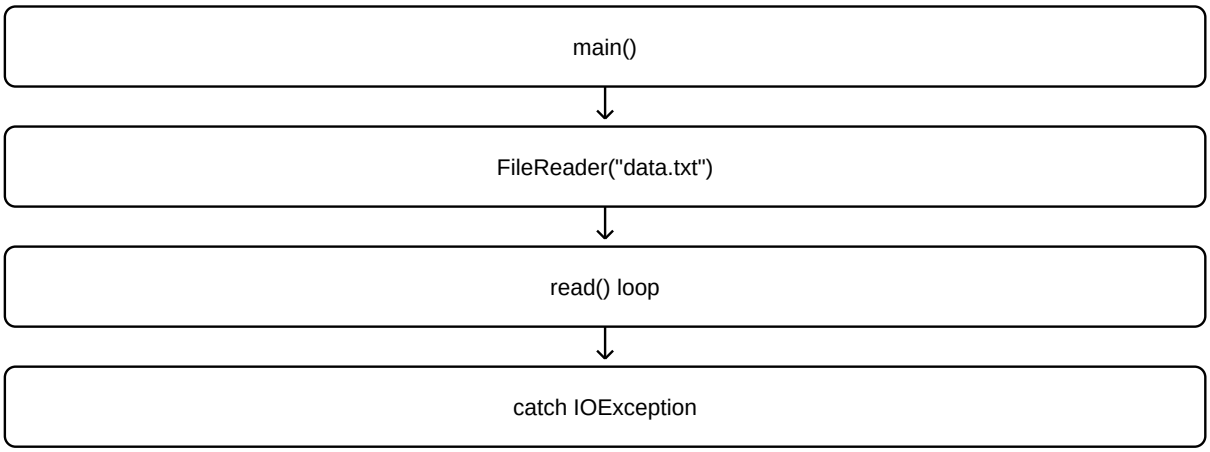
1. Open data.txt using FileReader.

2. Read and print characters until the end of the file.

3. Catch IOException if a reading error occurs.

4. UML Diagram (simplified)

Read from top to bottom. Boxes show the main class, method, action, or exception-handling step.



5. Source Code

```
import java.io.*;

public class IOExceptionExample {
    public static void main(String[] args) {

        try {
            FileReader file = new FileReader("data.txt");

            int ch;
            while ((ch = file.read()) != -1) {
                System.out.print((char) ch);
            }

            file.close();
        }
        catch (IOException e) {
            System.out.println("Input/Output error occurred");
        }
    }
}
```

6. Execution Output

```
Prints data.txt contents if readable.
If an IOException occurs: Input/Output error occurred
```

7. Result

The program reads and prints file contents and handles IOException. Normal output depends on data.txt.

Program 5: NullPointerExample

1. Aim

To demonstrate handling NullPointerException when using a null reference.

2. Requirements

Java Development Kit (JDK) and a text editor or IDE.

3. Algorithm

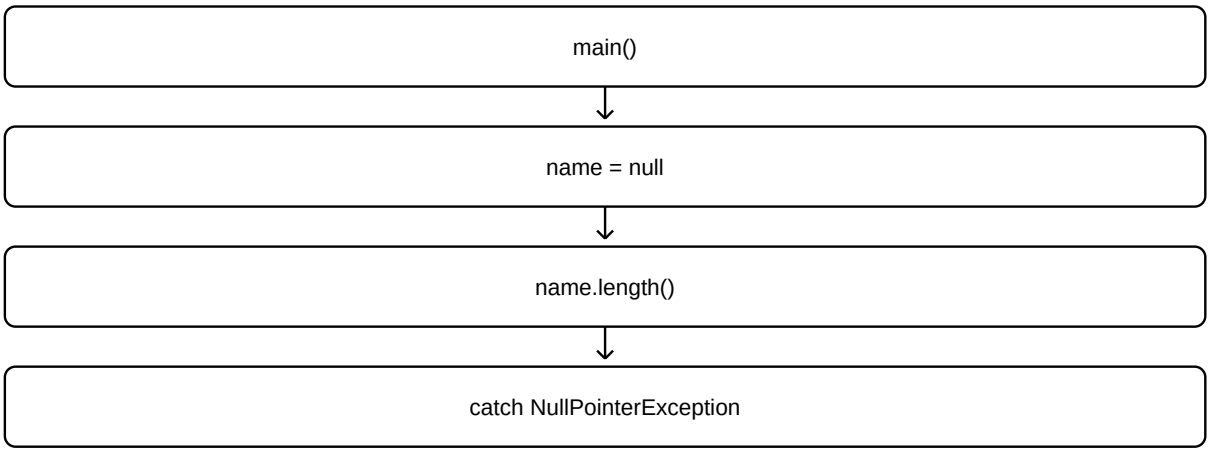
1. Assign null to the String variable name.

2. Attempt to call name.length() inside try.

3. Catch NullPointerException and print a message.

4. UML Diagram (simplified)

Read from top to bottom. Boxes show the main class, method, action, or exception-handling step.



5. Source Code

```
public class NullPointerExample {
    public static void main(String[] args) {

        String name = null;

        try {
            System.out.println(name.length());
        }
        catch (NullPointerException e) {
            System.out.println("Null value cannot be used");
        }
    }
}
```

6. Execution Output

```
Null value cannot be used
```

7. Result

The NullPointerException from calling length() on a null reference is caught and handled.

Program 6: NumberFormatExample

1. Aim

To demonstrate handling NumberFormatException when converting a non-numeric string to an integer.

2. Requirements

Java Development Kit (JDK) and a text editor or IDE.

3. Algorithm

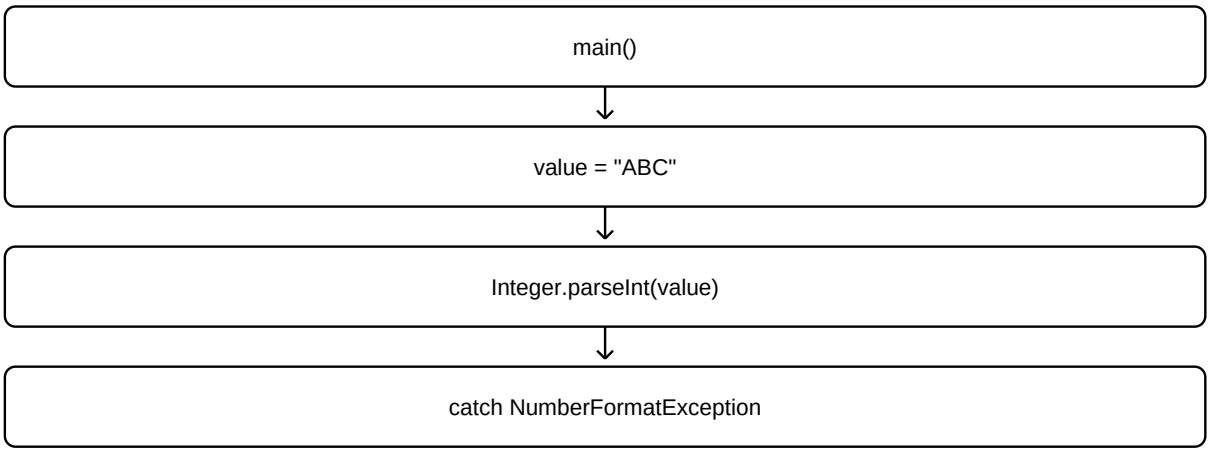
1. Assign the string ABC to value.

2. Try to convert value using Integer.parseInt().

3. Catch NumberFormatException and print an error message.

4. UML Diagram (simplified)

Read from top to bottom. Boxes show the main class, method, action, or exception-handling step.



5. Source Code

```
public class NumberFormatExample {
    public static void main(String[] args) {

        String value = "ABC";

        try {
            int number = Integer.parseInt(value);

            System.out.println("Number: " + number);
        }
        catch (NumberFormatException e) {
            System.out.println("Invalid number format");
        }
    }
}
```

6. Execution Output

```
Invalid number format
```

7. Result

The program catches NumberFormatException when ABC cannot be converted to an integer.

Program 7: StringIndexExample

1. Aim

To demonstrate handling StringIndexOutOfBoundsException when accessing an invalid string index.

2. Requirements

Java Development Kit (JDK) and a text editor or IDE.

3. Algorithm

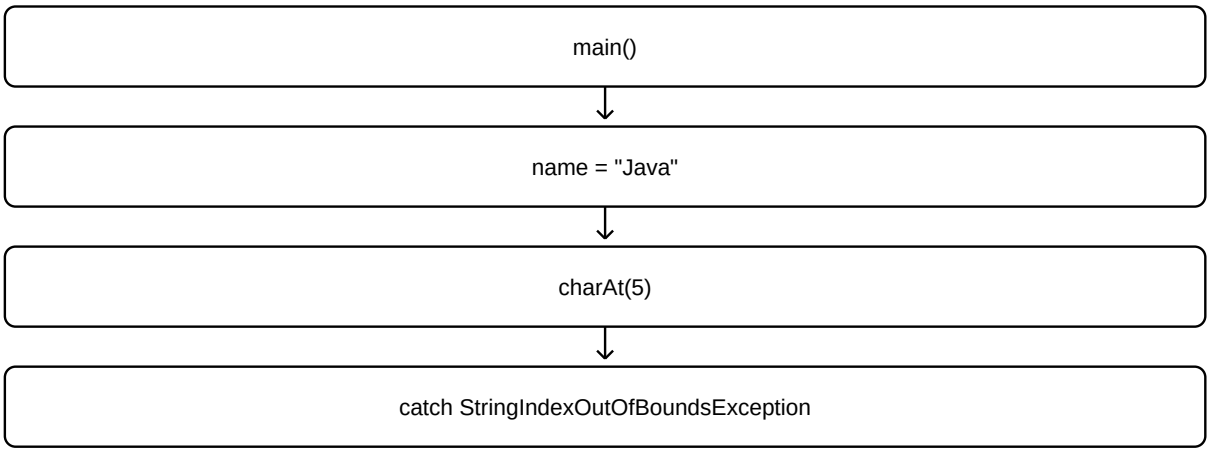
1. Initialize name with the string Java.

2. Attempt to print the character at index 5.

3. Catch StringIndexOutOfBoundsException because the index is invalid.

4. UML Diagram (simplified)

Read from top to bottom. Boxes show the main class, method, action, or exception-handling step.



5. Source Code

```
public class StringIndexExample {
    public static void main(String[] args) {

        String name = "Java";

        try {
            System.out.println(name.charAt(5));
        }
        catch (StringIndexOutOfBoundsException e) {
            System.out.println("String index is out of range");
        }
    }
}
```

6. Execution Output

```
String index is out of range
```

7. Result

The invalid string index is handled by catching StringIndexOutOfBoundsException.

Program 8: ClassCastExample

1. Aim

To demonstrate ClassCastException caused by casting an object to an incompatible subclass.

2. Requirements

Java Development Kit (JDK) and a text editor or IDE.

3. Algorithm

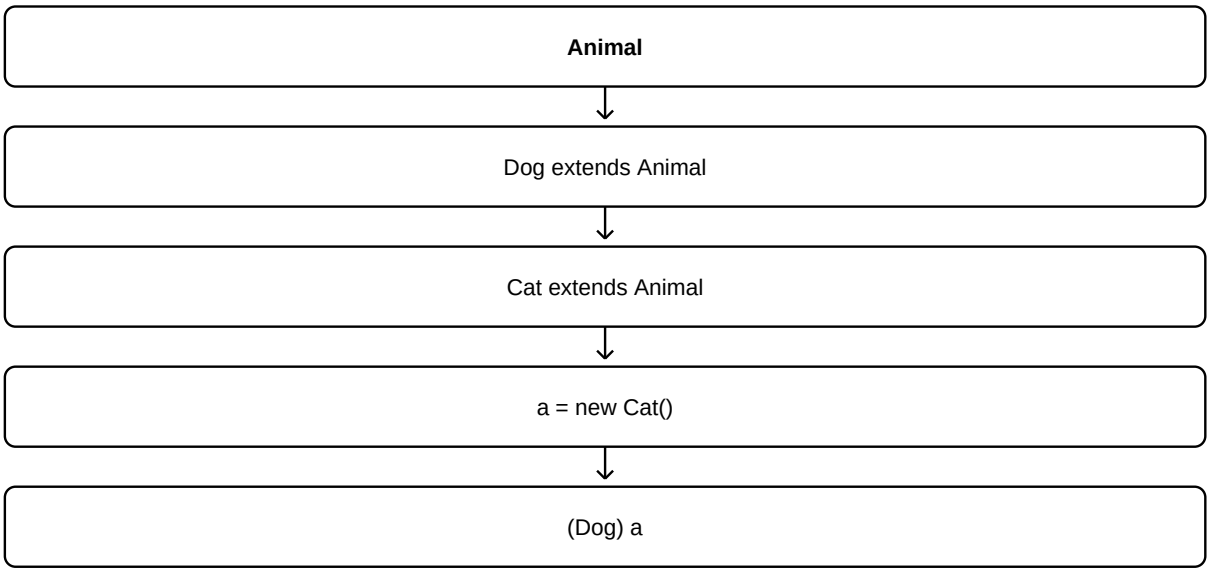
1. Define Animal as the parent class and Dog and Cat as subclasses.

2. Create a Cat object and store it in an Animal reference.

3. Attempt to cast that reference to Dog; the cast fails at runtime.

4. UML Diagram (simplified)

Read from top to bottom. Boxes show the main class, method, action, or exception-handling step.



5. Source Code

```
class Animal {
}

class Dog extends Animal {
}

class Cat extends Animal {
}

public class ClassCastExample {
    public static void main(String[] args) {

        Animal a = new Cat();

        Dog d = (Dog) a;

        System.out.println("Dog object created");
    }
}
```

6. Execution Output

```
Throws java.lang.ClassCastException at: Dog d = (Dog) a;
The message varies by Java version. "Dog object created" is not printed.
```

7. Result

The cast fails at runtime because the object is a Cat, not a Dog.

Program 9: IllegalArgumentExample

1. Aim

To demonstrate throwing and handling IllegalArgumentException when a method receives an invalid argument.

2. Requirements

Java Development Kit (JDK) and a text editor or IDE.

3. Algorithm

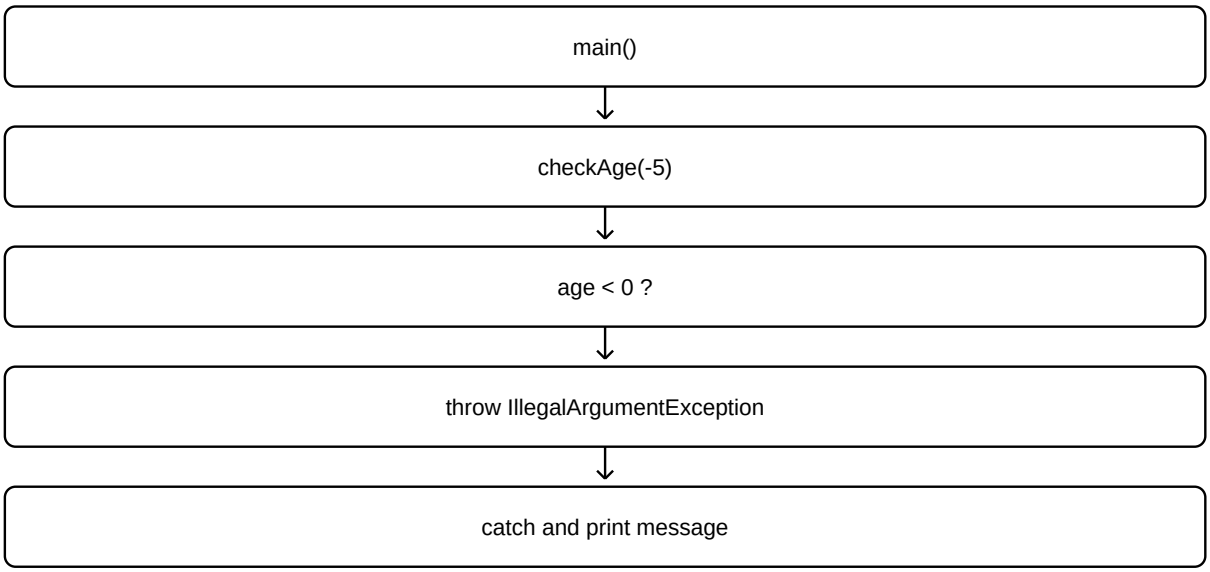
1. Call checkAge() with -5.

2. If age is negative, throw IllegalArgumentException.

3. Catch the exception in main() and print its message.

4. UML Diagram (simplified)

Read from top to bottom. Boxes show the main class, method, action, or exception-handling step.



5. Source Code

```
public class IllegalArgumentExample {

    static void checkAge(int age) {

        if (age < 0) {
            throw new IllegalArgumentException("Age cannot be negative");
        }

        System.out.println("Valid Age: " + age);
    }

    public static void main(String[] args) {

        try {
            checkAge(-5);
        }
        catch (IllegalArgumentException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

6. Execution Output

```
Exception: Age cannot be negative
```

7. Result

The method rejects a negative age by throwing IllegalArgumentException, which is handled in main().